Unix - Worksheet

This worksheet will guide you through using a Unix terminal and practising the available commands that you'd use throughout your course! Before attempting this worksheet, it is highly advised you have seen the accompanying lecture beforehand. If you missed it, watch the recording on the WCOMP101 Moodle page (or at least look through the lecture slides)

Section 0 - Connecting to Raptor

As explained in the lectures, Raptor is our own Kent Unix server for all Computing students, running Ubuntu Linux. You can follow this worksheet with your own Unix machine if you prefer but, for the sake of consistency, we will be assuming you'll be using Raptor.

Instructions

1. Open your terminal of choice:
   - Windows: Right-click on the Windows icon in the task bar, select "Terminal" from the popped-up list
   - Mac: Press the keys "command ⌘" and space on your keyboard at the same time (⌘ + space), type "Terminal.app" and hit enter
2. A command prompt window should appear. It may look slightly different depending on your system but will perform the same as the interface is text-based from here
3. Now we will connect to Raptor, type:

   ```
   ssh abc12@raptor.kent.ac.uk
   ```

   Replacing `abc12` with your Kent ID, and then hit enter. If it prompts you for whether you trust this connection or a similar message, press `y` or type `yes` and hit enter
4. It will prompt you for your password, enter it, and then hit enter again. Don't be alarmed if it doesn't look like you're typing anything here, that's normal for the terminal when entering passwords
5. Done! You should now be on Raptor via command line

Section 1 - Directories

Introduction

The Unix file system is organised hierarchically, like folders on your computer. Your **home directory** (`~`) is your personal space where you can create files and folders. We'll learn the essential commands for navigating, creating, and managing directories.

Task 1: Current Directory and Creating Your Workspace

You should now see a terminal entry ready for a command:

```
abc12@raptor [~] $
```

As a reminder, the first part is your username, then the machine's name, your current directory (in square brackets), and the dollar sign (`$`) shows where you can type your command. The `~` (tilde) represents your home directory.

1. Let's find out where your home directory actually is by typing the following command that will print out the working directory:

   ```
   pwd
   ```

   This stands for **Print Working Directory**.

2. For the purposes of today, we're going to create a directory where all of our tasks can take place so we don't create random files everywhere in your home directory.

   Create a new directory called "unix_learning" by using:

   ```
   mkdir unix_learning
   ```

   mkdir stands for **Make Directory**. You've just created a new folder!

3. Now let's enter the new directory we created by using the change directory command:

   ```
   cd unix_learning
   ```

   cd stands for **Change Directory**. Notice your terminal prompt should now show:

   ```
   abc12@raptor [~/unix_learning] $
   ```

4. Run `pwd` again to confirm we've moved into the new directory:

   ```
   pwd
   ```

Task 2: Navigating Directories

Now that we're in our workspace, let's learn how to navigate around the file system.

1. Create a few more directories to explore navigation. We'll create three subdirectories:

```
mkdir projects
mkdir documents
mkdir scripts
```

2. Move into the "projects" directory:

```
cd projects
```

   Your prompt should now show you're in `~/unix_learning/projects`.

3. Go back up one directory level (to the parent directory) using `..`:

```
cd ..
```

   You're now back in `unix_learning`. The `..` always refers to the parent directory.

4. Go back to your home directory quickly using:

```
cd ~
```

   Your prompt should show `~` again. The `~` is a shortcut to your home directory from anywhere.

5. Now navigate back to your unix_learning directory using the full path:

```
cd ~/unix_learning
```

Task 3: Listing Directory Contents

Let's see what's inside our directories using the `ls` command.

1. Make sure you're in the unix_learning directory:

```
cd ~/unix_learning
```

2. List the contents of the current directory:

```
ls
```

   You should see the three directories we created: `projects`, `documents`, `scripts`.

3. Now let's use `ls` with some options to see more detailed information. The `-l` option shows a "long" format with more details:

```
ls -l
```

   This shows the file permissions (first column), ownership, file size, date modified, and the name. We'll learn more about permissions later.

4. The `-a` option shows all files, including hidden ones (files starting with a dot):

```
ls -a
```

You'll see `.` (current directory) and `..` (parent directory) as well as any hidden files.

5. Combine both options to see detailed information including hidden files:

```
ls -la
```

6. The `-h` option shows file sizes in human-readable format (KB, MB, GB instead of bytes):

```
ls -lh
```

7. You can also list the contents of a different directory without entering it:

```
ls projects
```

This shows what's inside the projects directory from where you are now.

Task 4: Copying, Moving, and Removing Directories

Now let's practice manipulating directories.

1. Make sure you're in unix_learning:

```
cd ~/unix_learning
```

2. Copy the "projects" directory and all its contents to a new directory called "projects_backup":

```
cp -r projects projects_backup
```

The `-r` option means "recursive" — it copies the directory and everything inside it.

3. Move (rename) the "projects_backup" directory to "archive":

```
mv projects_backup archive
```

mv stands for **Move**. When moving to the same directory, it renames the directory. Check with `ls` to confirm.

4. Move the "archive" directory inside the "documents" directory:

```
mv archive documents/archive
```

Now it's nested inside documents. Check with `ls documents/`.

5. Move it back out to the main unix_learning directory:

```
mv documents/archive archive
```

6. Remove the empty "scripts" directory:

```
rmdir scripts
```

rmdir stands for **Remove Directory**, but it only works if the directory is empty.

7. If you want to remove a directory that has contents, use `rm` with the `-r` option:

```
rm -r archive
```

Warning: This will delete the directory and everything inside it permanently!

Challenge Task - Section 1

Create a directory structure that represents a project organisation. You should:

1. Create a main directory called "my_project" inside unix_learning
2. Inside my_project, create three subdirectories: "source", "build", and "output"
3. Inside "source", create two more subdirectories: "main" and "utils"
4. List the entire structure using `ls -la my_project` and nested `ls` commands
5. Create a backup of the entire "my_project" directory called "my_project_v1"
6. Move "my_project_v1" into a new directory called "backups"
7. Verify your final structure by listing the unix_learning directory

---

Section 2 - Files

Introduction

Files are the core of Unix. Most configuration, code, and data is stored as text files. In this section, you'll learn how to create, view, edit, search, and delete text files.

Task 1: Creating Text Files

There are several ways to create text files in Unix. Let's explore them.

1. Make sure you're in the unix_learning directory:

```
cd ~/unix_learning
```

2. The simplest way to create an empty file is using `touch`:

```
touch myfile.txt
```

This creates an empty file called "myfile.txt". Verify it was created:

```
ls
```

3. You can also create a file and add content to it at the same time using `echo` and redirection (`>`):

```
echo "Hello, this is my first file!" > greeting.txt
```

The `>` symbol redirects the output of the echo command into a file. If the file doesn't exist, it creates it.

4. Create another file with multiple lines. This time, we'll use a special syntax that lets you type multiple lines:

```
cat > story.txt << EOF
Once upon a time, there was a student learning Unix.
They discovered the power of the command line.
And they lived happily ever after.
EOF
```

Press Enter after pasting this entire block.

5. Verify all three files were created:

```
ls -l
```

Task 2: Viewing File Contents

Now let's look at what's inside our files.

1. View the entire contents of a file using `cat` (concatenate):

```
cat greeting.txt
```

This displays the entire file contents on the screen.

2. Use `less` to view a file with scrolling capabilities (useful for large files):

```
less story.txt
```

Press `Space` to scroll down, `b` to scroll back, and `q` to quit.

3. View only the first few lines of a file using `head`:

```
head -n 2 story.txt
```

This shows the first 2 lines. The `-n` option specifies the number of lines.

4. View only the last few lines using `tail`:

```
tail -n 1 story.txt
```

This shows the last 1 line.

Task 3: Editing Files

Now let's edit the files we created. We'll use `nano`, which is a beginner-friendly text editor.

1. Open the greeting.txt file in the nano editor:

```
nano greeting.txt
```

2. You'll see the file contents. Use arrow keys to move around and edit the text. Add a new line at the end that says "This is an edit!"
3. To save your changes, press `Ctrl+O` (the letter O, not zero), then press Enter to confirm the filename.
4. To exit nano, press `Ctrl+X`.
5. Verify your changes:

```
cat greeting.txt
```

Task 4: Searching Within Files

The `grep` command is powerful for finding specific text within files.

1. Create a file with multiple lines for searching:

```
cat > animals.txt << EOF
cat
dog
cat
bird
cat
fish
EOF
```

2. Search for the word "cat" in the file:

```
grep "cat" animals.txt
```

   This shows all lines containing "cat".

3. Count how many times "cat" appears (lines containing it):

```
grep -c "cat" animals.txt
```

   The `-c` option counts matching lines.

4. Show the line numbers where "cat" appears:

```
grep -n "cat" animals.txt
```

5. Search for lines that do NOT contain a specific word using `-v`:

```
grep -v "cat" animals.txt
```

Task 5: Copying, Moving, and Deleting Files

Let's practice file manipulation.

1. Copy a file to create a backup:

   ```
   cp greeting.txt greeting_backup.txt
   ```

   cp stands for **Copy**. Now you have two identical files.

2. Verify both files exist:

   ```
   ls -l greeting*
   ```

   The `*` is a wildcard that matches any characters. This shows all files starting with "greeting".

3. Move (rename) a file:

   ```
   mv story.txt my_story.txt
   ```

   The file "story.txt" is now called "my_story.txt".

4. Move a file to a different directory:

   ```
   mv greeting_backup.txt documents/
   ```

   The file is now inside the documents directory. Check:

   ```
   ls documents/
   ```

5. Move it back:

   ```
   mv documents/greeting_backup.txt .
   ```

   The `.` means the current directory.

6. Delete a file (be careful, this is permanent!):

   ```
   rm myfile.txt
   ```

7. Delete multiple files matching a pattern:

   ```
   rm greeting*
   ```

   This deletes all files starting with "greeting".

Challenge Task - Section 2

Create a mini project documentation system:

1. Create a directory called "docs" inside unix_learning
2. Inside docs, create three files: "README.txt", "CHANGELOG.txt", and "NOTES.txt"

3. Add content to README.txt describing a simple project (use echo or nano)
4. Add at least 3 version entries to CHANGELOG.txt
5. Add several random notes to NOTES.txt
6. Search through all three files for the word "version" using grep and redirect the output to a file called "search_results.txt"
7. Copy the entire docs directory to "docs_backup"
8. List all files in both directories to verify they're identical

---

Section 3 - Permissions

Introduction

Unix is a multi-user operating system, and file permissions control who can read, write, and execute files. Understanding permissions is crucial for security and file management.

Similarly, there are some commands requiring elevated privileges which we'll have to skip as, due to security reasons, you won't have access to these commands on Raptor.

Understanding Permission Basics

Permissions in Unix are represented by three types:

- **Read (r)**: Can view file contents
- **Write (w)**: Can modify or delete file contents
- **Execute (x)**: Can run the file as a program

These apply to three categories:

- **User (u)**: The file owner
- **Group (g)**: Users in the file's group
- **Other (o)**: Everyone else

Task 1: Viewing Permissions

1. Make sure you're in the unix_learning directory:

```
cd ~/unix_learning
```

2. View files with their permissions using `ls -l`:

```
ls -l
```

3. Let's examine what the permission string means. Create a test file first:

```
touch test_file.txt
```

4. View it with permissions:

```
ls -l test_file.txt
```

   You'll see something like: `-rw-r--r--`

   Breaking this down:

   - First character `-` = regular file (d = directory, l = link)
   - Characters 2-4 `rw-` = owner permissions (read, write, no execute)
   - Characters 5-7 `r--` = group permissions (read only)
   - Characters 8-10 `r--` = other permissions (read only)

Task 2: Changing Permissions with Symbolic Notation

The `chmod` (change mode) command modifies permissions. We can use symbolic notation:

1. Add execute permission for the owner:

```
chmod u+x test_file.txt
```

   `u+x` means: for the user, add execute permission.

2. Check the permissions changed:

```
ls -l test_file.txt
```

   You should now see `-rwxr--r--`.

3. Remove write permission from the group:

```
chmod g-w test_file.txt
```

   `g-w` means: for the group, remove write permission.

4. Add write permission for others:

```
chmod o+w test_file.txt
```

5. Add read permission to everyone:

```
chmod a+r test_file.txt
```

   `a` means "all" (user, group, and other).

6. Remove all permissions except for the owner (owner can still do everything):

```
chmod go-rwx test_file.txt
```

   This removes all permissions from group and others.

Task 3: Changing Permissions with Numeric Notation

Permissions can also be represented as numbers:

- **r (read)** = 4
- **w (write)** = 2
- **x (execute)** = 1

Add these up for each category:

- 7 = 4+2+1 = rwx (all permissions)
- 5 = 4+1 = r-x (read and execute)
- 4 = r-- (read only)
- 0 = --- (no permissions)

The format is: `chmod [user][group][other] filename`

1. Give the owner all permissions, group read-only, others nothing:

   ```
   chmod 740 test_file.txt
   ```

2. Verify:

   ```
   ls -l test_file.txt
   ```

   You should see `rwxr-----`.

3. Give everyone read and write (but not execute):

   ```
   chmod 666 test_file.txt
   ```

4. Give everyone full permissions:

   ```
   chmod 777 test_file.txt
   ```

5. Standard permissions for scripts (owner can execute, others can read):

   ```
   chmod 755 test_file.txt
   ```

Task 4: Changing File Ownership

***Important Note**: On Raptor, you won't have permission to change file ownership, but you can still learn the command syntax.*

The `chown` (change owner) command changes who owns a file. Note: You can usually only change ownership if you're the root user, but let's see the syntax.

1. View the current owner of a file:

   ```
   ls -l test_file.txt
   ```

The third column shows the owner (usually your username).

2. The command to change ownership would be:

```
sudo chown newuser test_file.txt
```

Note: This requires administrator (sudo) privileges. As a regular user, you'll see a permission denied error, which is expected.

3. You can change the group of a file (if you're in that group):

```
sudo chown :groupname test_file.txt
```

Challenge Task - Section 3

Create a system for managing script permissions:

1. Create a directory called "scripts" in unix_learning
2. Create three files in scripts: "backup.sh", "deploy.sh", and "readme.txt"
3. Initially set all files to be readable by everyone but not executable
4. Make "backup.sh" and "deploy.sh" executable only by the owner
5. Make "readme.txt" readable by everyone but writable only by the owner
6. Create a summary of the permissions by running `ls -l scripts/` and save the output to "permissions_summary.txt"
7. Verify that only the .sh files have the execute permission by using `ls -l` and looking for the x in the permission string

---

Section 4 - Piping and Redirection

Introduction

One of Unix's most powerful features is the ability to chain commands together using pipes and to redirect output to files. This allows you to build complex operations from simple commands.

Task 1: Understanding Input, Output, and Redirection

Every command has three standard streams:

- **stdin (Standard Input)**: Where input comes from (usually keyboard)
- **stdout (Standard Output)**: Where output goes (usually terminal screen)
- **stderr (Standard Error)**: Where error messages go

1. Create a test file with some data:

```
cat > numbers.txt << EOF
3
1
4
1
5
9
2
10
6
EOF
```

2. Create another file with names:

```
cat > names.txt << EOF
alice
bob
charlie
alice
diana
EOF
```

Task 2: Redirecting Output to Files

The `>` operator redirects stdout to a file (overwrites if file exists).

1. Redirect the output of `ls` to a file:

```
ls > directory_listing.txt
```

2. View the file:

```
cat directory_listing.txt
```

3. Try running the command again:

```
ls > directory_listing.txt
```

   The file is overwritten with the new output.

4. To append to a file instead of overwriting, use `>>`:

```
echo "This is an additional line" >> directory_listing.txt
```

5. Verify the line was added:

```
cat directory_listing.txt
```

Task 3: Using Pipes to Chain Commands

The `|` (pipe) operator takes the stdout of one command and sends it as stdin to another command.

1. List the unix_learning directory and count the number of items:

```
ls | wc -l
```

   `wc` is the word count command. `-l` counts lines. We piped the output of `ls` into `wc -l`.

2. Sort the contents of names.txt:

```
cat names.txt | sort
```

3. Sort and remove duplicates:

```
cat names.txt | sort | uniq
```

4. Count unique names:

```
cat names.txt | sort | uniq | wc -l
```

5. Sort numbers:

```
cat numbers.txt | sort
```

   Note: This sorts them as text, so 10 comes before 2.

6. Sort numbers numerically using `sort -n`:

```
cat numbers.txt | sort -n
```

Task 4: Searching and Filtering with Pipes

The `grep` command is especially powerful when used with pipes.

1. List all files and show only those with ".txt" in the name:

```
ls | grep ".txt"
```

2. List files with details and filter for files larger than 0 bytes:

```
ls -l | grep -v "^total"
```

   `-v` inverts the match (shows lines that don't match).

3. Create a file with programming languages:

```
cat > languages.txt << EOF
Python
JavaScript
Java
Rust
Go
Python
C++
Rust
EOF
```

4. Find and count occurrences of "Python":

```
grep "Python" languages.txt | wc -l
```

5. Show unique languages (sorted):

```
cat languages.txt | sort | uniq
```

Task 5: Redirecting Input from Files

The `<` operator redirects a file as input to a command.

1. Use a file as input to the `sort` command:

```
sort < numbers.txt
```

This is equivalent to `cat numbers.txt | sort`, but the data goes directly to sort.

2. Count the number of lines in a file:

```
wc -l < names.txt
```

3. Combine input and output redirection:

```
sort < numbers.txt > sorted_numbers.txt
```

This reads from numbers.txt, sorts it, and writes to sorted_numbers.txt.

Task 6: Combining Pipes and Redirection

You can use pipes and redirection together for powerful data processing.

1. Find all files in the current directory, list them with details, filter for readable files, and save to a file:

```
ls -l | grep "r--" > readable_files.txt
```

2. Count how many files are readable:

```
ls -l | grep "r--" | wc -l
```

3. Create a sorted, unique list of text files and save it:

```
ls *.txt | sort | uniq > text_files_list.txt
```

4. Search for a pattern in multiple files and count results:

```
grep "the" *.txt | wc -l
```

Challenge Task - Section 4

Create a comprehensive data analysis pipeline:

1. Create a file called "data.txt" with at least 10 lines containing numbers (1-100, some repeated)
2. Create a file called "keywords.txt" with at least 5 different words
3. Using pipes and redirection, perform these tasks:
   - Sort the numbers in data.txt and save to "sorted_data.txt"
   - Find unique numbers and save to "unique_numbers.txt"
   - Count how many unique numbers exist (save the count to "count.txt")
   - List all .txt files, count them, and save the result with a label to "file_summary.txt"
4. Create a pipeline that searches for keywords in your files and saves results to "search_results.txt"
5. Verify all output files were created correctly by listing and viewing them

---

Section 5 - Running Code (Java, Python, and Rust)

Introduction

The Unix command line is where programmers compile, run, and manage code projects. In this section, you'll learn how to work with Java, Python, and Rust from the terminal.

Task 1: Running Python Code

Python is an interpreted language, meaning you can run it directly without compilation.

1. Create a simple Python script:

```
cat > hello.py << EOF
print("Hello from Python!")
print("This is running on the Unix command line.")

name = input("What is your name? ")
print(f"Nice to meet you, {name}!")
EOF
```

2. Run the Python script:

```
python3 hello.py
```

   Follow the prompt to enter your name.

3. Create a Python script that does some calculations:

```
cat > calculator.py << EOF
numbers = [1, 2, 3, 4, 5]
total = sum(numbers)
average = total / len(numbers)

print(f"Numbers: {numbers}")
print(f"Sum: {total}")
print(f"Average: {average}")
EOF
```

4. Run it:

```
python3 calculator.py
```

5. To see all Python scripts in your directory:

```
ls -l *.py
```

Task 2: Python Virtual Environments & Installing and Using Python Packages

Virtual environments let you isolate project dependencies so they don't conflict with system-wide packages. Here we can use Python's package manager called `pip` that lets you install additional libraries.

1. Create a virtual environment called "myenv":

```
python3 -m venv myenv
```

2. Activate the virtual environment:

```
source myenv/bin/activate
```

   Notice your terminal prompt changes to show `(myenv)` at the beginning.

3. Inside the virtual environment, install a package (requests - for making web requests):

```
pip3 install requests
```

4. Create a Python script that uses the installed package:

```
cat > check_status.py << EOF
import requests

# This would normally check a website's status
# For now, just demonstrate the import worked
print("Requests library successfully imported!")
print(f"Requests version: {requests.__version__}")
EOF
```

5. Run it:

```
python3 check_status.py
```

6. Deactivate the virtual environment when done:

```
deactivate
```

The `(myenv)` prefix disappears from your prompt.

Task 3: Compiling and Running Java Code

Java is a compiled language, meaning you must compile code before running it.

1. Create a simple Java program:

```
cat > HelloWorld.java << EOF
public class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello from Java!");
        System.out.println("Running on the Unix command line.");
    }
}
EOF
```

Important: Java filenames must match the class name exactly, including capitalisation.

2. Compile the Java code:

```
javac HelloWorld.java
```

This creates a file called `HelloWorld.class`.

3. Run the compiled Java program:

```
java HelloWorld
```

Note: You use the class name without the .class extension.

4. Create a more complex Java program:

```
cat > Calculator.java << EOF
public class Calculator {
    public static void main(String[] args) {
        int a = 10;
        int b = 5;

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("a + b = " + (a + b));
        System.out.println("a * b = " + (a * b));
    }
}
EOF
```

5. Compile and run:

```
javac Calculator.java
java Calculator
```

6. List the compiled files:

```
ls -l *.class
```

Task 4: Compiling and Running Rust Code

Rust is a compiled language known for safety and performance.

1. Create a simple Rust program:

```
cat > main.rs << EOF
fn main() {
    println!("Hello from Rust!");
    println!("Running on the Unix command line.");
}
EOF
```

2. Compile the Rust code:

```
rustc main.rs
```

   This creates an executable file called `main`.

3. Run the compiled Rust program:

```
./main
```

   Note: You need to use `./` to run executables in the current directory.

4. Create a more complex Rust program:

```
cat > calculator.rs << EOF
fn main() {
    let a = 10;
    let b = 5;

    println!("a = {}", a);
    println!("b = {}", b);
    println!("a + b = {}", a + b);
    println!("a * b = {}", a * b);
}
EOF
```

5. Compile and run:

```
rustc calculator.rs
./calculator
```

Task 5: Managing Rust Projects with Cargo

For larger Rust projects, use Cargo, Rust's package manager and build system.

1. Create a new Rust project:

```
cargo new my_rust_project
```

   This creates a directory with the project structure.

2. Enter the project directory:

```
cd my_rust_project
```

3. View the project structure:

```
ls -la
```

   You'll see `src/main.rs` (your code) and `Cargo.toml` (project configuration).

4. Build the project:

```
cargo build
```

   This compiles your code and creates an executable in the `target/debug/` directory.

5. Run the project directly:

```
cargo run
```

   This builds and runs the project in one command.

6. Build an optimised release version:

```
cargo build --release
```

This creates a faster binary in `target/release/`.

7. Return to your working directory:

```
cd ..
```

Challenge Task - Section 5

Create a multi-language demonstration project:

1. Create a directory called "code_examples"
2. In "code_examples", create:
   - A Python script called "greet.py" that takes a name as input and prints a personalised greeting:

```
cat > code_examples/greet.py << EOF
name = input("Enter your name: ")
print(f"Hello, {name}! Welcome to the multi-language demo.")
EOF
```

   - A Java program called "MathDemo.java" that calculates and displays the area and perimeter of a rectangle:

```
cat > code_examples/MathDemo.java << EOF
import java.util.Scanner;

public class MathDemo {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("Enter length of rectangle: ");
        double length = scanner.nextDouble();
        System.out.print("Enter width of rectangle: ");
        double width = scanner.nextDouble();

        double area = length * width;
        double perimeter = 2 * (length + width);

        System.out.println("Area: " + area);
        System.out.println("Perimeter: " + perimeter);
    }
}
EOF
```

   - A Rust program called "fibonacci.rs" that calculates the first 10 Fibonacci numbers:

```
cat > code_examples/fibonacci.rs << EOF
fn main() {
    let mut a = 0;
    let mut b = 1;

    println!("First 10 Fibonacci numbers:");
    for _ in 0..10 {
        println!("{}", a);
        let temp = a;
        a = b;
        b = temp + b;
    }
}
EOF
```

3. Compile and run each program, verifying it works correctly
4. Create a text file called "run_log.txt" that documents what you ran and when
5. List all source files (.py, .java, .rs) and all compiled files (.class, executables) to show the difference
6. Create a summary file listing the command to compile and run each language

---

Section 6 - Shell Scripts

Introduction

Shell scripts are files containing Unix commands that execute one after another. They automate repetitive tasks and are essential for system administration and development workflows.

Task 1: Creating and Running Basic Shell Scripts

1. Create a simple shell script using nano:

   ```
   nano my_script.sh
   ```

   In the editor, type or paste the following:

   ```
   #!/bin/bash
   echo "Welcome to shell scripting!"
   echo "This script is running on the Unix command line."
   pwd
   ```

   The `#!/bin/bash` line (called a "shebang") tells Unix to run this script with the bash shell.

2. Save the file (`CTRL + O`) and exit nano (`CTRL + X`).
3. Make the script executable:

   ```
   chmod +x my_script.sh
   ```

Verify it's executable by checking permissions:

```
ls -l my_script.sh
```

You should see an `x` in the permissions.

4. Run the script:

```
./my_script.sh
```

Task 2: Using Variables in Shell Scripts

Variables store data that your script can use and manipulate.

1. Edit your shell script using nano:

```
nano my_script.sh
```

2. Add variables to that script:

```
PROJECT_NAME="My Unix Project"
VERSION="1.0"
AUTHOR="Student"
```

3. Use the variables in echo statements:

```
echo "Project: $PROJECT_NAME"
echo "Version: $VERSION"
echo "Author: $AUTHOR"
```

4. Save your script (`CTRL + O`) and exit nano (`CTRL + X`)
5. Then run it again:

```
./my_script.sh
```

6. Edit again using nano:

```
nano my_script.sh
```

7. Create variables in your script that command results are stored in:

```
CURRENT_DATE=$(date)
FILE_COUNT=$(ls -1 | wc -l)
CURRENT_USER=$USER

echo "Date: $CURRENT_DATE"
echo "Number of files: $FILE_COUNT"
echo "User: $CURRENT_USER"
```

8. Save and exit nano again, then run it:

```
./my_script.sh
```

Task 3: Using Conditional Statements (if/else)

Conditionals let your script make decisions.

1. Create a new script using nano:

```
nano file_checker.sh
```

2. Create a script with if statements:

```
#!/bin/bash

# Check if a file exists
FILENAME="test_file.txt"

if [ -f "$FILENAME" ]; then
    echo "File $FILENAME exists."
else
    echo "File $FILENAME does not exist."
fi

# Check if a directory exists
DIRNAME="documents"

if [ -d "$DIRNAME" ]; then
    echo "Directory $DIRNAME exists."
else
    echo "Directory $DIRNAME does not exist."
fi
```

   Inside the script, `-f` checks for a file and `-d` checks for a directory.

3. Save (`CTRL + O`) and exit (`CTRL + X`) nano
4. Run the script:

```
chmod +x file_checker.sh
./file_checker.sh
```

5. Create a script that compares numbers:

```
nano number_compare.sh
```

   In the editor, type or paste:

```
#!/bin/bash

NUM1=10
NUM2=5

if [ $NUM1 -gt $NUM2 ]; then
    echo "$NUM1 is greater than $NUM2"
elif [ $NUM1 -eq $NUM2 ]; then
    echo "$NUM1 equals $NUM2"
else
    echo "$NUM1 is less than $NUM2"
fi
```

Inside the script, `-gt` means greater than, `-eq` means equal to. Similarly, you can use `-lt` for less than. Save and exit nano.

6. Run it:

```
chmod +x number_compare.sh
./number_compare.sh
```

Task 4: Using Loops

Loops repeat commands multiple times.

1. Create a script with a for loop:

```
nano for_loop_demo.sh
```

In the editor, type or paste:

```
#!/bin/bash

echo "Counting from 1 to 5:"
for i in 1 2 3 4 5; do
    echo "Number: $i"
done

echo ""
echo "Listing files:"
for file in *.txt; do
    if [ -f "$file" ]; then
        echo "Found: $file"
    fi
done
```

Save and exit nano.

2. Run it:

```
chmod +x for_loop_demo.sh
./for_loop_demo.sh
```

3. Create a script with a while loop:

```
nano while_loop_demo.sh
```

In the editor, type or paste:

```
#!/bin/bash

echo "Countdown from 3:"
COUNT=3
while [ $COUNT -gt 0 ]; do
    echo "$COUNT..."
    COUNT=$((COUNT - 1))
done
echo "Blast off!"
```

Save and exit nano.

4. Run it:

```
chmod +x while_loop_demo.sh
./while_loop_demo.sh
```

Task 5: Creating Functions in Shell Scripts

Functions let you write reusable blocks of code.

1. Create a script with functions:

```
nano functions_demo.sh
```

In the editor, type or paste:

```
#!/bin/bash

# Define a function
greet() {
    echo "Hello! Welcome to this shell script."
}

# Define a function with parameters
add_numbers() {
    local sum=$(($1 + $2))
    echo "The sum of $1 and $2 is $sum"
}

# Call the functions
greet
add_numbers 5 3
add_numbers 10 20
```

Save and exit nano.

2. Run it:

```
chmod +x functions_demo.sh
./functions_demo.sh
```

3. Create a practical script that uses functions:

```
nano file_operations.sh
```

In the editor, type or paste:

```
#!/bin/bash

# Function to count files
count_files() {
    local count=$(ls -1 $1 | wc -l)
    echo "Directory $1 contains $count items"
}

# Function to show file details
show_details() {
    echo "Details of $1:"
    ls -lh $1
}

# Use the functions
count_files .
show_details .
```

Save and exit nano.

4. Run it:

```
chmod +x file_operations.sh
./file_operations.sh
```

Task 6: Practical Script Examples

1. Create a backup script:

```
nano backup.sh
```

In the editor, type or paste:

```
#!/bin/bash

# Create a backup of a directory
SOURCE_DIR="documents"
BACKUP_DIR="documents_backup_$(date +%Y%m%d_%H%M%S)"

if [ -d "$SOURCE_DIR" ]; then
    cp -r "$SOURCE_DIR" "$BACKUP_DIR"
    echo "Backup created: $BACKUP_DIR"
else
    echo "Source directory not found!"
fi
```

Save and exit nano.

2. Make it executable and run it:

```
chmod +x backup.sh
./backup.sh
```

3. Create a script that creates a directory structure for a project (taking the project name as a parameter):

```
nano create_project.sh
```

In the editor, type or paste:

```
#!/bin/bash

PROJECT_NAME="$1"

if [ -z "$PROJECT_NAME" ]; then
    echo "Usage: $0 <project_name>"
    exit 1
fi

echo "Creating project structure for $PROJECT_NAME..."

mkdir -p "$PROJECT_NAME/src"
mkdir -p "$PROJECT_NAME/build"
mkdir -p "$PROJECT_NAME/docs"
mkdir -p "$PROJECT_NAME/tests"

echo "Project structure created in ./$PROJECT_NAME"
echo "Structure:"
ls -la "$PROJECT_NAME"
```

Save and exit nano.

4. Run it:

```
chmod +x create_project.sh
./create_project.sh my_project
```

Challenge Task - Section 6

Create a comprehensive system administration script:

1. Create a script called "file_analyser.sh" that:
   - Searches through text files for a keyword (passed as a parameter)
   - Counts occurrences and unique occurrences
   - Lists which files contain the keyword
   - Uses loops and conditional statements
2. Create a script called "python_project_manager.sh" that:
   - Takes a project name as a parameter
   - Creates a project directory with subdirectories (src, docs, input, output)
   - Creates a main.py file in src that prints "Hello, !" in the src directory
   - Creates a README.md file in the project with project information
   - Creates a configuration file, config.json, that stores the project creation date
   - Uses functions for organisation
3. Test each script with appropriate parameters and verify the output

---

Additional Tips and Tricks

Using the Manual Command

The `man` command provides documentation for any command:

```
man ls
man cd
man chmod
```

Press `Space` to scroll down, `b` to go back, and `q` to quit.

Viewing Command History

See all commands you've typed:

```
history
```

Or search through history:

```
history | grep "command_name"
```

Use `!` to repeat commands:

```
!ls    # Repeats the last ls command
```

Clearing the Terminal

Clear the screen:

```
clear
```

Finding Files and Directories

Search for files by name:

```
find /path -name "filename"
```

Search in current directory:

```
find . -name "*.txt"
```

Helpful Keyboard Shortcuts

- `Ctrl+C` - Stop a running command
- `Ctrl+Z` - Suspend a running command
- `Ctrl+L` - Clear the screen (same as `clear`)
- `Ctrl+A` - Go to beginning of line
- `Ctrl+E` - Go to end of line
- `Tab` - Auto-complete file and directory names
- `Ctrl+R` - Search through command history
- ↑ Arrow - Previous command
- ↓ Arrow - Next command

Conclusion

You've now learned the fundamental Unix command-line skills! These concepts form the foundation of working with Unix and Linux systems. As you continue learning, you'll find that the command line becomes increasingly powerful and efficient for:

- Managing files and directories
- Processing and analyzing data
- Running and developing code
- Automating repetitive tasks
- System administration

- Scripting and automation

Keep practicing, experiment with combinations of commands, and don't be afraid to use `man` to explore commands you haven't seen before. The Unix command line is a vast and powerful tool—the more you use it, the more proficient you'll become!

Any hey, if you’re curious in going down this rabbit hole further, why not try a Linux distribution on your own machine?